

Exam: Node Multiway Cut Problem

Abstract

This report presents the implementation of three algorithms for the Node Multiway Cut problem: Isolating Cut Heuristic, a $(2 - \frac{2}{k})$ approximation algorithm for Multiway Cuts in Node weighted graphs and an improved Parameterized Algorithm for the Minimum Node Multiway Cut Problem. The algorithms have been evaluated on generated graph data types and the PACE 2018 Competition datasets to compare runtimes, quality of the cut from different algorithms and how well they scale based on increasing graph sizes and increasing number of terminals. The experiments also highlight the speedup that parallelization provides and how additional overhead affects algorithm performance. The practical benefits of the theoretical guarantees are discussed in the conclusions.

1 Introduction

The Multiway Cut Problem (MCP) is a graph partitioning and combinatorial optimization problem in computer science. Given a graph and a set of terminal nodes, a Node Multiway cut is a set of minimum vertices that leaves terminals disconnected from each other when removed from the graph. When there are only two terminals, the multiway cut problem is the same as the *minimum $s-t$ cut problem* and can be computed efficiently. The problem is NP-hard when there are three or more terminals, making it computationally expensive to find optimal solutions for large instances. Closely related variants of the problem are the minimum weighted Node Multiway Cut problem and the Edge Multiway cut problem, where the objective is to reduce the weight of the cut set. These variants are used in applications such as network isolation, graph-based image segmentation and VLSI

design, where minimizing separation cost is essential.

We study three algorithms that address the Minimum Node Multiway Cut problem. We begin with a greedy Heuristic based on the notion of Isolation Cuts [6] [2], adapted from the edge multiway cut formulation. We then consider a $(2 - 2/k)$ Approximation for Multiway Cuts in Node Weighted Graphs [4], adapted as unweighted by taking weight as 1. Finally we examine an Improved Parameterized Algorithm for the Minimum Node Multiway Cut Problem [1].

The algorithms are evaluated on a diverse set of generated graph instances and instances from the PACE 2018 competition.

The paper is structured as follows: Section 2 formally introduces the Node Multiway Cut problem, theoretical foundations of the algorithms and other preliminary concepts. Section 3 presents the three algorithm approaches and their implementation. Sections 4 and 5 present experimental results and draw conclusions.

2 Preliminaries

In this section, we begin with the formal definition of the Multiway Cut problem, the max-flow min-cut theorem, BFS and Edmonds-Karp algorithm, the Isolating Cut Heuristic and the the multicommodity flow algorithm.

2.1 Node Multiway Cut Problem

Let $G = (V, E)$ be an undirected graph and $T \subseteq V$ be a set of terminals $s_1, s_2, \dots, s_k$. A node multiway cut is a set of non-terminal nodes whose removal disconnects each terminal from all others. Usually, the problem is to find a cut of minimum weight in a node weighted graph. In our case, we take the weight of all nodes to be 1 in order to find a node cut of minimum cardinality. In the rest of

the paper, this problem will be referred to as the Multiway Cut Problem or MCP.

2.2 Max-Flow Min-Cut theorem

Given a graph $G = (V, E)$ with capacities c :

- The maximum flow problem seeks a feasible flow f from a vertex s to a vertex t that maximizes the total value of the flow.
- An s - t cut is a partition $(S, V \setminus S)$ such that $s \in S$ and $t \notin S$. The capacity of a cut:

$$c(S, V \setminus S) = \sum_{\substack{(u,v) \in E \\ u \in S, v \notin S}} c(u, v)$$

By Max-Flow Min-Cut Theorem, the maximum value of an s - t flow equals the minimum capacity of any s - t cut

2.3 Multicommodity Flow

Given a commodity associated with a pair of terminals s_i and s_j , designated as *source* and *sink*, the multicommodity flow problem [4] maximizes the sum of flows of the commodities subject to:

- Flow conservation: a commodity is conserved at each node except the source and sink for that commodity.
- Throughput constraints: the sum of the flows of all commodities through a node cannot exceed the weight of the node.

This problem can be formulated as a linear program with $O(k^2|E|)$ variables and constraints, whose optimal value provides a lower bound on the minimum multiway cut and can be found in polynomial time.

2.4 Breadth-First search

The Breadth-First Search is a graph traversal algorithm that explores vertices level by level. Given a graph $G = (V, E)$ and a source node s , BFS visits all nodes at distance 1 from s , then at distance 2 and so on. Hence the runtime of BFS is $O(E)$.

2.5 Edmonds-Karp Algorithm

The Edmonds-Karp algorithm solves the maximum flow problem in directed graphs. It repeatedly augments flow along shortest augmenting paths found using *BFS* in the residual graph. Each augmentation increases the shortest path length or saturates at least one edge. The overall total runtime is $O(VE^2)$

- Number of augmentations is $O(VE)$
- Each BFS takes $O(E)$

2.6 Isolation cut Heuristic

An isolation cut [2] for any terminal $s_i \in S$ is defined as minimum cut of edges $E_i \subseteq E$ separating s_i from all other terminals. It is computed by connecting all terminals $s_j \in T, j \neq i$ to a super-sink t with infinite capacity edges and then returning a minimum s_i - t cut via max-flow (Section 2.2).

3 Algorithm & Implementation

This section introduces the greedy isolation cut heuristic [6], the $(2 - \frac{2}{k})$ approximation [4] and the improved parameterized exact approach [1] for solving the Node Multiway Cut Problem. We first describe the algorithms along with the pseudo code and then go in detail about their implementation.

3.1 Greedy Heuristic Algorithm

The greedy heuristic approach uses the isolating cuts 2.6 of the terminals. For each $s_i \in S$, [6] an isolating cut set of edges $E_i \subseteq E$ is defined, whose removal separates the terminal s_i from all other terminals. To compute a minimum weight isolating cut for s_i , all other terminals are connected with infinite capacity to a new vertex t , which behaves like a sink. a maximum flow is then computed between s_i and t . By the Max-Flow Min-Cut theorem, the resulting minimum s - t cut is

a minimum weight isolating cut. Hence, the isolating cut heuristic reduces the MCP to k max-flow computations. The algorithm then outputs a union of the $k - 1$ lightest cuts. This heuristic yields a $(2 - \frac{2}{k})$ approximation. The over all runtime is $O(k \cdot T_{\max\text{-flow}})$. If Edmonds-Karp [3] algorithm the chosen to compute the max-flows, the runtime would be $O(KVE^2)$

Algorithm 1 Isolation Cut Heuristic

Require: Graph $G(V, E)$, Terminals $T \subseteq V$

Ensure: Multiway cut A

```

1: for each terminal  $t_i \in T$  do
2:   Construct auxiliary graph  $G_i$ 
3:   Add new node  $sink$ 
4:   for each terminal  $t \in T \setminus \{t_i\}$  do
5:     Add edge  $(t, sink)$  with  $\infty$  capacity
6:   end for
7:   edgeCut  $E_i \leftarrow \text{MinCut}(G_i, t_i, sink)$ 
8: end for
9:  $w(E_1) \leq w(E_2) \leq \dots \leq w(E_k)$  {Assume without
   loss of generality}
10:  $A \leftarrow \bigcup_{i=1}^{k-1} E_i$ 
11: return  $A$ 
12:
13: function MinCut( $G_i, t_i, sink$ )
14:  $G_r \leftarrow \text{BuildResidualNetwork}(G_i)$ 
15:  $(f, G_r) \leftarrow \text{EdmondsKarp}(G_i, t_i, sink)$ 
16:  $R \leftarrow \text{BFS}(G_r, t_i)$ 
17:  $E \leftarrow \{(u, v) \in E(G_i) \mid u \in R, v \notin R\}$ 
18: return  $E$ 
19:
20: function EdmondsKarp( $G_i, t_i, sink$ )
21:  $G_r \leftarrow \text{BuildResidualNetwork}(G_i)$ 
22:  $f \leftarrow 0$ 
23: while  $\exists$  augmenting path  $P$  from  $t_i$  to sink in
    $G_r$  do
24:    $\Delta \leftarrow \min\{c_r(e) \mid e \in P\}$ 
25:   Augment flow along  $P$  by  $\Delta$ 
26:   Update  $G_r$ 
27:    $f \leftarrow f + \Delta$ 
28: end while
29: return  $(f, G_r)$ 

```

Since we consider the node multiway cut, we adapt this approach by interpreting the removal

of an edge as the removal of its endpoints. Hence, the nodes incident to the selected edges are added to the cutset.

The parallelization is performed over the set of terminals $s_1 \dots s_k$. Each processor is assigned one terminal and computes its isolating cut via a max-flow. Assuming there are k processors available, the total work remains $O(K \cdot T_{\max\text{-flow}})$. The parallel time is reduced to $O(T_{\max\text{-flow}})$, as all k computations can be executed concurrently. Hence, by distributing these computations across, the algorithm achieves a parallel speedup of up to k , ignoring overheads.

3.2 Approximation Algorithm

Algorithm 2 $(2 - \frac{2}{k})$ Approximation

Require: Graph $G(V, E)$, Terminals $T \subseteq V$

Ensure: Multiway cut A

```

1:
2: compute optimal dual_lp_solution()
3: for each terminal  $s_i \in S$  do
4:   Compute region  $S_i$ 
5:   Compute boundary  $\Gamma(S_i)$ 
6:   Compute boundary  $\Gamma^{1/2}(S_i)$ 
7:    $j \leftarrow \arg\max_{i \in \{1, \dots, k\}} w(\Gamma^{1/2}(S_i))$ 
8: end for
9:  $C \leftarrow \bigcup_{i=1}^k \Gamma(S_i) \setminus \Gamma^{1/2}(S_j)$ 
10: return  $C$ 

```

The linear program for the Multicommodity problem 2.3 can be rewritten from the point of multiway cuts: let f_i denote the flow along each of the paths p_i between two distinct terminals.

(P1) Primal:

$$\begin{aligned}
& \text{maximize} && \sum_i f_i \\
& \text{subject to} && \sum_{i: v \in p_i} f_i \leq w_v && \forall v \in V \setminus T \\
& && f_i \geq 0 && \forall \text{ paths } p_i
\end{aligned}$$

(P2) Dual:

$$\begin{aligned}
& \text{minimize} && \sum_{v \in V \setminus T} d_v w_v \\
& \text{subject to} && \sum_{v \in p_i} d_v \geq 1 && \forall \text{ paths } p_i \\
& && d_v \geq 0 && \forall v \in V \setminus T
\end{aligned}$$

The dual is the assignment of a non-negative length label d_v to non-terminal nodes $v \in V \setminus T$. These node lengths induce a distance function on the graph. The constraints of the dual LP ensure that the distance between any pair of terminals is at least one. This means that any path connecting two terminals must pass through nodes with positive length. For any integral solution, where $d_v \in \{0, 1\}$, corresponds to a node multiway cut: nodes with $d_v = 1$ intersect every path between terminals, and thus their removal separates all terminals. The objective function $\sum_{v \in V \setminus T} d_v w_v$ corresponds to the weight of the cut.

(P3) Dual Reformulation:

$$\begin{aligned}
\min & \sum_{v \in V \setminus T} d_v w_v \\
\text{s.t.} & y_{v,j} \leq y_{u,j} + d_v && \forall (u,v) \in E, \forall j = 1, \dots, k \\
& y_{s_j,j} = 0 && \forall j = 1, \dots, k \\
& y_{s_i,j} \geq 1 && \forall i, j = 1, \dots, k, i \neq j \\
& d_v \geq 0 && \forall v \in V \setminus T
\end{aligned}$$

The dual LP formulation P2 contains a path for every path between terminals, resulting in an exponential number of constraints. To obtain a polynomial-size formulation, the LP is reformulated using variables $y_{u,j}$ that represents distance from a terminal s_j to node u .

It is shown in [5] that the dual LP admits an optimal half-integral solution, i.e., $d_v \in \{0, \frac{1}{2}, 1\}$ for all $v \in V \setminus T$. We use this structural property to interpret the solution in terms of regions and boundaries and construct a feasible multiway cut.

The implementation 2 first solves the polynomial-size dual LP reformulation and

obtains node-length variables d_v together with distance variables $y_{u,j}$. For each terminal s_j , the region S_j is defined as the set of nodes with $y_{u,j} = 0$, nodes at zero distance from s_j . The boundary of each region is then computed by considering nodes adjacent to the region but not contained in any region.

Using the half-integral structure of the LP solution, boundary nodes are classified into unique boundaries and shared boundaries. Unique boundary nodes ($\Gamma^{1/2}(S_i)$) are those nodes that only belongs to the boundary of one terminal and have $d_v = 0.5$. Shared boundary nodes $\Gamma(S_i)$ are those nodes are in the boundaries of at least 2 terminals and have $d_v = 1$. Any path between two terminals must pass through either a $\Gamma(S_i)$ node or at least two nodes from $(\Gamma^{1/2}(S_i))$, ensuring that boundary nodes intersect all such paths. Thus, removing boundary nodes disconnects all terminal pairs. Finally, the algorithm constructs the cut as the union of all region boundaries, excluding the heaviest unique half-boundary. By excluding this, we remove a significant portion of the total boundary weight while still preserving the separation of all terminal pairs. The resulting set remains a valid multiway cut, and its weight can be bounded by $(2 - \frac{2}{k}) \cdot \text{maximum flow}$. We refer to Theorem 5 in [4] for the full proof of correctness and the approximation bound.

3.3 Exact Algorithm

For an exact approach for the Node Multiway cut problem, we implement a parameterized Node Multiway Cut solution [1] with a runtime of $O(n^3 k^4 4^k)$. The algorithm systematically explores feasible separators of size at most k using a bounded search tree where each branching step reduces the parameter or increases the minimum cut value.

The algorithm first checks if an edge connects two terminal sets as no valid separator exists because terminals cannot be part of the node cut. If a non-terminal w is adjacent to two terminal sets, it must belong to the multiway cut. We then delete w from the graph and recursively find a

cut on $(G - w, k - 1)$. Then m_1 is computed as the minimum V-cut separating T_1 from $\bigcup_{j=2}^{\ell} T_j$ [2.2]. If $k \leq m_1$, then size of the cut separating T_1 from the rest of the terminals is more than k , hence no separator of size bounded by k can exist for the terminal sets. If $m_1 = 0$ and $\ell = 2$, T_1 is already separated from $T_2, \dots, T_{\ell}$, and the remaining terminal sets need to be separated. Hence, the algorithm recursively solves the instance $(G, \{T_2, \dots, T_{\ell}\}, k)$.

At step 6, no edge connects two different terminal sets, no non-terminal has neighbors in multiple terminal sets and $0 < m_1 \leq k$. By condition 3 in [1], there must be a non-terminal u adjacent to T_1 .

Compute m as the new minimum cut between $T'_1 \leftarrow T_1 \cup \{u\}$ and $\bigcup_{j=2}^{\ell} T_j$. If $m = m_1$, then adding u to T_1 does not change the cut value. By Theorem 3.2 in [1], the instance remains equivalent, and we recursively solve $(G, \{T_1, \dots, T_{\ell}\}, k)$.

Otherwise, if $m > m_1$, we branch on u . In the first case, u is included in the separator, and we recursively solve $(G - u, \{T_1, \dots, T_{\ell}\}, k - 1)$. In the second case, u is excluded from the separator, and by Lemma 3.3, this reduces to solving the instance $(G, \{T_1, \dots, T_{\ell}\}, k)$.

The algorithm branches on a vertex u , either including it in the separator (reducing k) or excluding it (increasing the cut constraint). This induces a bounded search tree, where each branching step decreases the parameter k , resulting in at most 4^k leaves. Each path, dominated by minimum V cuts which $O(n^3 k)$ time. Therefore, the overall running time is $O(n^3 k \cdot 4^k)$ and is of the form $f(k) \cdot \text{poly}(n)$.

3.4 Implementation Details

The implementation uses C++ and integrates the GLPK solver [5] for the LP formulation.

The parallel greedy algorithm uses OpenMP to parallelize the computation of isolating cut for each terminal. The loop is parallelized using `#pragma omp parallel for` with dynamic scheduling to balance the workload, as different max-flow instances may have varying runtimes.

Algorithm 3 Parameterized Exact Algorithm

Require: $G(V, E)$, $T = \{T_1, \dots, T_{\ell}\}$, k

Ensure: Multiway cut L or No

```

1: if edge endpoints in two terminal sets then
2:   return No
3: end if
4: if  $w \notin T$  has two neighbors in two different
   terminal sets then
5:   return  $\{w\} \cup \text{NMC}(G - w, \{T_1, \dots, T_{\ell}\}, k - 1)$ 
6: end if
7:  $m_1 \leftarrow \text{minV-cut between } T_1 \text{ and } \bigcup_{j=2}^{\ell} T_j$ 
8: if  $m_1 > k$  then
9:   return No
10: end if
11: if  $m_1 = 0$  and  $\ell = 2$  then
12:   return  $\emptyset$ 
13: end if
14: if  $m_1 = 0$  and  $\ell > 2$  then
15:   return  $\text{NMC}(G, \{T_2, \dots, T_{\ell}\}, k)$ 
16: end if
17: Pick  $u \notin T$  adjacent to  $T'_1$ , set  $T_1 \leftarrow T_1 \cup \{u\}$ 
18: if  $\text{min V-cut}(T'_1, \bigcup_{j=2}^{\ell} T_j) = m_1$  then
19:   return  $\text{NMC}(G, \{T'_1, \dots, T_{\ell}\}, k)$ 
20: end if
21:  $S \leftarrow \{u\} \cup \text{NMC}(G - u, \{T_1, \dots, T_{\ell}\}, k - 1)$ 
22: if  $S \neq \text{No}$  then
23:   return  $S$ 
24: else
25:   return  $\text{NMC}(G, \{T'_1, T_2, \dots, T_{\ell}\}, k)$ 
26: end if

```

For thread safety, the following design choices were made:

- The graph is passed as read-only during parallel execution. each max-flow computation constructs its own residual graph to avoid any shared mutable state between threads.
- The containers (`isolatingCuts` and `cutSizes`) are preallocated, and each thread writes only to its assigned index.
- A shared atomic flag is used to enforce the time limit. The synchronization points were minimized to maximize parallel efficiency.

In the merging phase, the conversion from edge cuts to node cuts is parallelized by distributing the processing of selected cuts across threads.

In the implementation of the exact algorithm, the graph itself is not explicitly modified during recursion. A state structure represents changes in the graph for each recursive subproblem. This state maintains vertices that are deleted from the graph and included in the cut, the current terminal groups and locked vertices that cannot be included in the multiway cut. Operations such as $(G - u)$ and $T_1 \leftarrow T_1 \cup u$ are realized by updating the state, while the underlying graph remains unchanged. This design avoids repeated graph reconstruction and enables efficient exploration of the search tree.

The evaluation framework includes a graph generator for graph type grid. Unit test for max-flow and integration tests for the algorithms ensure correctness. The command line interface support experimental execution. Results from benchmarks are stored in CSV files and are later used for analysis.

4 Experimental Evaluation

In this section, we evaluate the performance of the greedy heuristic, $(2 - \frac{2}{k})$ approximation approach and the parametrized exact approach for solving the Multiway cut problem.

4.1 Data and Hardware

Experiments were conducted on a server at the University of Konstanz equipped with an AMD Ryzen Threadripper 3970X processor featuring 32 physical cores and 64 hardware threads (maximum clock speed 3.7 GHz) and 128 MiB of L3 cache and on an Intel Core Ultra 7 55U CPU (7 cores) running at 2.10 GHz with 32GB RAM. Three datasets were used for evaluation:

- Generated graphs: 185 grid graph instances of varying sizes, with up to 2000 vertices and 3910 edges. For each graph size, we consid-

ered terminal sets of sizes up to 50 and generated 5 instances per configuration.

- PACE 2018 Small: 200 graphs with up to 19,083 vertices, 35,636 edges and 24 terminals
- PACE 2018 Medium: 200 graphs with up to 36,415 vertices, 145,635 edges, and 16,808 terminals
- PACE 2018 Large: 200 graphs with up to 235,686 vertices, 366,093 edges, and 17,832 terminals

Each algorithm was executed with the following configurations: all four algorithms ran once on all 200 instances of the Small PACE instances (240s timeout), 199 Medium PACE instances (420s timeout). For 150 Large instances, only the greedy variants were evaluated once each (600s timeout). On the generated grid graphs, On the generated graph benchmark, all four algorithms were executed once on each of the 185 generated instances using a time limit of 120

Performance of the algorithms was measured using four key metrics:

- Cut size: number of nodes in the computed cut.
- Cut validity: whether all terminals are separated after removing the cut nodes.
- Execution time: measured in milliseconds.
- Timeout status: whether the algorithm exceeded the time limit.

The performance is evaluated by comparing the cut sizes output by the different algorithms, graph topology and runtimes.

4.2 Experimental Results

Our experimental evaluation yielded comprehensive data on the performance characteristics of all four algorithms across the test datasets. Below we analyze these results from multiple perspectives, focusing on solution quality, runtime

performance, behavior across different graph types, and scaling properties

Table 1 summarizes the average structural properties of the benchmark datasets used in the experiments. It reports the average number of terminals, graph density, and edge-to-node ratio over all benchmarked instances. These statistics serve as reference values for interpreting the runtime and solution-quality trends discussed below.

Dataset	Avg. $ T $	Avg. Density	Avg. $ E / V $
Generated	15.7	0.030	1.858
PACE S	19.4	0.048	10.517
PACE M	524.6	0.010	1.721
PACE L	319.1	0.043	12.944

Table 1: Average structural properties over all benchmarked instances in each dataset.

The solved percentages in Table 2 refer to runs that returned a valid multiway cut. Unsolved runs are explained by timeouts or by instances in which terminals were directly adjacent, making the instance infeasible for the algorithm configurations.

4.2.1 Runtime Performance

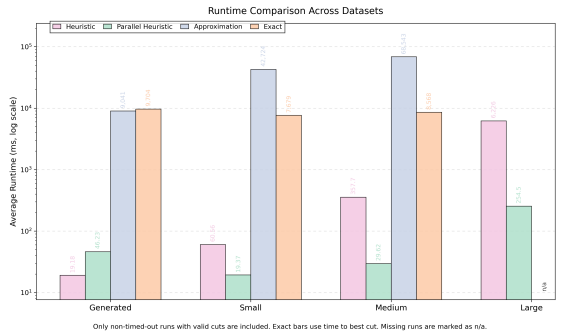


Figure 1: Runtime comparison across datasets.

Figure 1 compares the runtime behavior of the algorithms across the generated graphs and

the PACE datasets. On the PACE benchmark, the parallel heuristic is consistently the fastest among the algorithms that scale to all tested tracks. Its mean runtime over valid runs is 19.4ms on PACE Small, 29.6ms on PACE Medium, and 254.5ms on PACE Large. The corresponding runtimes of the sequential heuristic are 60.6ms, 357.7ms, and 6226.3ms, showing a clear advantage for parallel execution on the server hardware. The LP-based approximation is substantially slower, with mean runtimes of 42.7s on Small and 68.5s on Medium, reflecting the overhead of solving a linear program for each instance.

For the exact algorithm, the plotted runtime corresponds to the time at which the best cut was found rather than the total execution time until termination. In addition, timed-out runs are excluded from the runtime plot. This is important when interpreting the exact results, since the reported values summarize only successful runs. Hence, the hardest exact cases are mostly missing from the average.

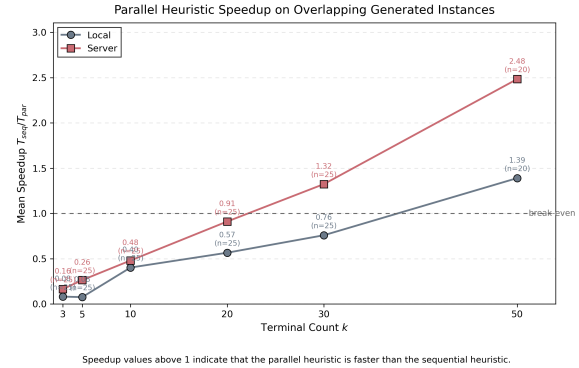


Figure 2: Runtime comparison on different hardware configurations.

To isolate the effect of hardware on parallelization, an additional comparison of sequential and parallel heuristics on the set of generated grid instances is done on both the local machine and the UniKonstanz server. We report speedup as the ratio between sequential and parallel runtime. On the 7-core Intel laptop, the parallelized

heuristic is slower than the sequential version due to thread management and synchronization overhead. However, on the 32-core UniKonstanz server, the parallel implementation significantly outperforms the sequential version, demonstrating that parallelization is beneficial only when sufficient computational resources are available.

4.2.2 Solution Quality

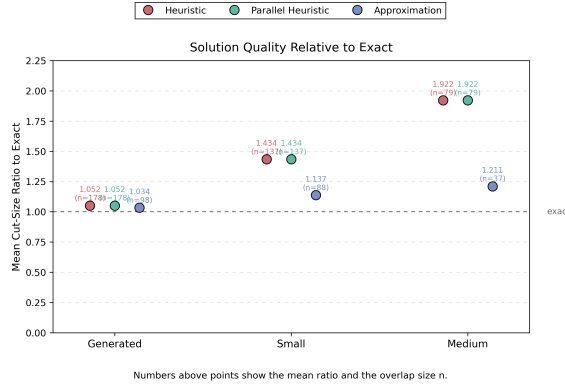


Figure 3: Runtime comparison across datasets.

The solution quality is measured as ratio between the cut size produced by heuristic or approximation algorithms and the cut size produced by the exact algorithm. The results show that the LP-based approximation consistently produces solutions closer to optimal compared to the heuristic. While both methods perform similarly on small generated instances, the gap increases with instance sizes. The heuristic variants exhibit a significant degradation in quality, reaching ratios closer to 2 on medium instances where as the approximation remains within a much tighter range.

The heuristic and its parallel variant yields identical solution quality. Given the time limit, exact and ILP solutions could not be computed for large instances and the analysis is restricted to instances where optimal solutions are available. This introduces a bias towards small instances but still highlights the relative robustness of the

approximation approach.

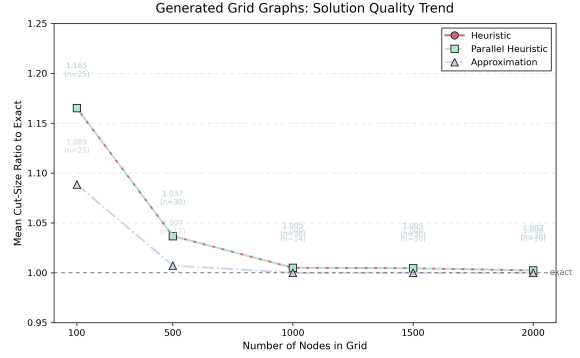


Figure 4: solution quality on generated grid graphs

On the generated grid instances, solution quality tends to improve as the grid size increases to 2000 nodes. In particular, the cut-size ratios of both the heuristic and the approximation move closer to 1 on larger grids, meaning that their solutions become closer to the exact solution. This suggests that, for this regular graph family, increasing the number of nodes does not make the heuristics less accurate. Instead, both methods approach exact-quality performance on the larger grid instances.

4.2.3 Scaling

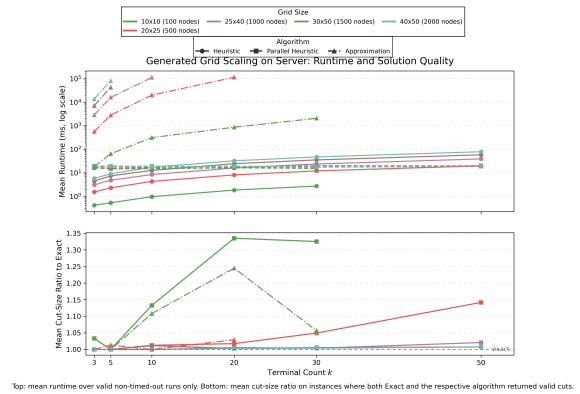


Figure 5: scaling on grid graphs.

Scalability is assessed on the generated grid benchmark, where grid size and terminal count k vary in a controlled manner. As grid size and terminal count increase, runtime grows for all algorithms, but much more sharply for the approximation than for the heuristic variants. The heuristic remains close to optimal on these controlled instances. On the common subset where both approximation and exact returned valid cuts, its mean cut-size ratio is 1.056, compared to 1.031 for the approximation. Thus, on the generated grids, the heuristic scales better computationally, while the approximation has a small advantage in solution quality.

The same trade-off is visible on the PACE benchmark 1. The PACE tracks are structurally heterogeneous, they are therefore used mainly to evaluate robustness. The heuristic variants scale better computationally among the three datasets, with runtimes in the millisecond range and no observed timeouts. The approximation is much more expensive, requiring 42.7s on Small and 68.5s on Medium on average over valid runs. However, the approximation achieves better solution quality on the tracks where exact comparisons are available. Its mean cut-size ratio is 1.137 versus 1.434 for the heuristic on PACE Small, and 1.211 versus 1.922 on PACE Medium. These results should be interpreted as evidence of robustness across increasingly difficult benchmark datasets rather than for scaling.

5 Discussion and Conclusion

The evaluation of the greedy heuristic [2], its parallel variant, the LP-based $(2 - \frac{2}{k})$ approximation [5], and the $O(n^3 k 4^k)$ exact algorithm [1] on the PACE 2018 benchmark datasets highlights their differing behavior across the different data sets and instance sizes.

The experimental results are consistent with the theoretical complexity of the algorithms. The isolation-cut heuristic, with polynomial worst-case running time $O(kVE^2)$ when Edmonds–Karp [3] is used for the max-flow computations,

is the fastest method in practice and remains feasible on all tested tracks. Its parallel variant further improves runtime on multi-core systems and provides the best overall computational scalability in our experiments. The LP-based approximation is also polynomial-time, but substantially more expensive in practice because it requires solving a large linear program whose size grows with both the graph and the number of terminals. As a result, it offers better solution quality but at significantly higher runtime cost. The exact algorithm, with running time $O(n^3 k \cdot 4^k)$, performed better than the approximation and was practical only for the small track and a subset of the medium instances.

Based on these results, the parallel heuristic provides the most practical overall approach within the scope of our experiments, combining low runtime with good scalability across the tested datasets. The approximation remains the stronger option when solution quality is the primary objective, while the exact algorithm is mainly useful as a reference method on smaller instances. These conclusions are specific to the datasets, graph sizes, hardware, and time limits considered in this study, and performance may differ on other graph families or under different experimental conditions.

References

- [1] Jianer Chen, Yang Liu, and Songjian Lu. An improved parameterized algorithm for the minimum node multiway cut problem. *Algorithmica*, 55(1):1–13, 2009.
- [2] Elias Dahlhaus, David S. Johnson, Christos H. Papadimitriou, Paul D. Seymour, and Mihalis Yannakakis. The complexity of multi-terminal cuts. *SIAM Journal on Computing*, 23(4):864–894, 1994.
- [3] Jack Edmonds and Richard M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *J. ACM*, 19(2):248–264, April 1972.

- [4] Naveen Garg, Vijay V. Vazirani, and Mihalis Yannakakis. Multiway cuts in node weighted graphs. *Journal of Algorithms*, 50(1):49–61, 2004.
- [5] GNU Project. GNU Linear Programming Kit (GLPK). <https://www.gnu.org/software/glpk/>, 2024. Accessed: 2026-03-24.
- [6] University of Illinois at Urbana-Champaign. Multiway cut and related problems. https://courses.grainger.illinois.edu/cs598csc/sp2009/lectures/lecture_7.pdf. Lecture notes, accessed 2026.

A Additional Plots

Dataset	Algorithm	Instances	Attempted	Solved	Solved (%)	Timeouts	Edge Between*
Generated	Heuristic	185	185	183	98.9	0	3
Generated	Parallel Heuristic	185	185	183	98.9	0	3
Generated	Approximation	185	185	97	52.4	80	3
Generated	Exact	185	183	177	96.7	0	6
Small	Heuristic	200	200	147	73.5	0	53
Small	Parallel Heuristic	200	200	147	73.5	0	53
Small	Approximation	200	200	88	44.0	59	53
Small	Exact	200	147	137	93.2	8	2
Medium	Heuristic	199	199	79	39.7	0	120
Medium	Parallel Heuristic	199	199	79	39.7	0	120
Medium	Approximation	199	199	37	18.6	42	120
Medium	Exact	199	79	79	100.0	0	0
Large	Heuristic	150	150	111	74.0	0	39
Large	Parallel Heuristic	150	150	111	74.0	0	39
Large	Approximation	150	0	0	n/a	0	0
Large	Exact	150	0	0	n/a	0	0

Table 2: Outcome summary per dataset and algorithm. Percentages are computed over attempted runs.

* Edge Between is inferred from CSV rows where the algorithm was attempted, did not time out, did not produce a valid cut, and has no reported cut size.